

Anexo - QA automatizado con agente de OpenClaw



”Porque el club es de los socios, y la gestión es de SocioUnido”

Plataforma inteligente que centraliza la gestión administrativa de los clubes, optimizando el uso de espacios compartidos, previniendo la morosidad y asegurando los accesos al estadio.

Alumno	Correo electrónico	Padrón
Ascencio, Felipe Santino	fascencio@fi.uba.ar	110675
Ghosn, Lautaro Gabriel	lghosn@fi.uba.ar	106998
Guerrero, Martín	mguerrero@fi.uba.ar	107774
Zielonka, Axel	azielonka@fi.uba.ar	110310

Jefe de Cátedra Mg. Ing. Carlos Fontela cfontela@fi.uba.ar

Tutor Mg. Ing. Gabriel Piñeiro gpineiro@fi.uba.ar

Cotutor Ing. Agustín Perrotta aperrotta@fi.uba.ar

Índice

1. QA automatizado con agente de OpenClaw	3
1.1. Introducción y objetivos	3
1.2. Uso de la herramienta OpenClaw	3
1.3. Definición de tareas y directrices del agente	3
1.4. Resultados obtenidos	4
1.4.1. Aplicación móvil PWA (Socios)	4
1.4.2. Aplicación móvil PWA (Trabajadores)	4
1.4.3. Gateway	5
1.4.4. Microservicio de accesos	5
1.4.5. Microservicio de analíticas	5
1.4.6. Microservicio de autenticación	6
1.4.7. Microservicio de bot conversacional (PLN)	6
1.4.8. Microservicio de gestión del club (Core)	6
1.4.9. Microservicio de pagos	7
1.4.10. Panel administrativo web	7
1.5. Acciones tomadas	8
1.5.1. Cifrado de datos en reposo y gestión segura de credenciales	8
1.5.2. Auditoría de dependencias y prevención de ataques de cadena de suministro	8
1.5.3. Sanitización de entradas, protección de interfaces y prevención de fuga de datos	8
1.5.4. Gestión de concurrencia y robustez de infraestructura	8
1.5.5. Protección criptográfica contra ataques de análisis de tiempo y suplantación	8
1.5.6. Fortalecimiento de los contratos de interfaz para interoperabilidad con IA	9
1.6. Conclusión	9

1. QA automatizado con agente de OpenClaw

1.1. Introducción y objetivos

En el marco del desarrollo de SocioUnido, alcanzar la más alta calidad en los estándares de ciberseguridad y de interoperabilidad se ha planteado como un objetivo fundamental e innegociable. En la actualidad, el ecosistema tecnológico experimenta un crecimiento exponencial en el desarrollo de herramientas de Inteligencia Artificial que interactúan directamente con plataformas y productos web. Por este motivo, hacemos un énfasis especial en garantizar la interoperabilidad de nuestro sistema con futuras IAs. Para que SocioUnido se mantenga a la vanguardia de estos avances, resulta clave que la arquitectura, las interfaces y los endpoints estén preparados no solo para usuarios humanos, sino también para integrarse de manera fluida, lógica y segura con agentes autónomos de terceros.

1.2. Uso de la herramienta OpenClaw

Para llevar a cabo una validación rigurosa de estos estándares, implementamos **OpenClaw**, un agente de Inteligencia Artificial diseñado específicamente para el testing automatizado y el QA (Aseguramiento de la calidad) integral de repositorios de código.

La adopción de OpenClaw se justifica por su capacidad innovadora para realizar auditorías exhaustivas sobre grandes volúmenes de código estructural. A diferencia de las herramientas de análisis estático tradicionales, OpenClaw comprende el contexto del código, identifica vulnerabilidades lógicas complejas y evalúa la preparación del sistema para la automatización, actuando como un auditor experto dedicado exclusivamente a las necesidades de SocioUnido.

1.3. Definición de tareas y directrices del agente

La implementación técnica del agente se basó en un marco de instrucciones precisas (prompting) que define su rol como *Experto en QA y Ciberseguridad*. Las directrices configuradas para su análisis auditan estrictamente tres ejes fundamentales:

- **Seguridad de la implementación:** Se instruye al agente a realizar una búsqueda exhaustiva de vulnerabilidades, haciendo foco en los estándares de la industria (como el OWASP Top 10) y en la prevención de vectores de ataque tanto típicos como atípicos.
- **Prevención de fuga de datos:** El agente debe asegurar rigurosamente que el código fuente no exponga credenciales, contraseñas, tokens de acceso ni información privada de los usuarios o de la institución. Este punto es considerado crítico, dada la naturaleza del sistema y su enfoque en ser comercializado a terceros.
- **Preparación para IA y automatización:** Se evalúa meticulosamente si la estructura de los datos y los endpoints del sistema están diseñados garantizando una interacción sin fricciones con futuras automatizaciones e Inteligencias Artificiales externas.

Para optimizar la legibilidad y eficiencia de la auditoría, se le exige al agente un formato de salida estricto. El reporte debe omitir el código limpio y detallar únicamente los archivos que presentan problemas o áreas de mejora, indicando para cada caso: el nombre del archivo, el problema detectado, la justificación de su impacto y la propuesta concreta de solución. En caso de no detectar fallas, el agente está instruido para emitir una única declaración certificando que la implementación cumple con todos los estándares definidos.

1.4. Resultados obtenidos

A continuación, se detalla el análisis exhaustivo de seguridad realizado por OpenClaw sobre los cinco repositorios principales que conforman la arquitectura de SocioUnido.

1.4.1. Aplicación móvil PWA (Socios)

- **Fuga de información sensible:** Se identificó que ante fallos inesperados de la aplicación, el componente encargado de capturar errores exponía la traza completa del sistema (stack trace) directamente en la pantalla del usuario, revelando la estructura interna del código fuente.
- **Manipulación de rutas API:** Se detectó una concatenación directa e insegura de parámetros en las peticiones HTTP enviadas por los servicios del cliente, dejando abierta la posibilidad de alterar las rutas de la interfaz de programación (Path Traversal) o enviar solicitudes malformadas.
- **Gestión de encabezados de autorización:** El módulo central de peticiones HTTP enviaba encabezados de autenticación con valores nulos o indefinidos cuando el usuario navegaba de forma anónima, generando un comportamiento ambiguo en los servidores.
- **Exposición de datos personales y tokens de notificación:** El contexto global de autenticación registraba en la consola del navegador información personal identificable del socio junto con los tokens del servicio de notificaciones, vulnerando la privacidad del usuario.
- **Configuración de entorno e integración de pagos:** Se detectó el hardcodeo de claves de prueba para la pasarela de pagos.
- **Metadatos del documento:** El archivo raíz de la interfaz mantenía el atributo de idioma configurado en inglés para una aplicación desarrollada íntegramente en español, lo que afectaba la accesibilidad y la interpretación de agentes externos.

1.4.2. Aplicación móvil PWA (Trabajadores)

- **Fuga de información sensible:** Se identificó que ante fallos inesperados de la aplicación, el componente encargado de capturar errores exponía la traza completa del sistema directamente en la pantalla del usuario, revelando la estructura interna del código fuente.
- **Gestión de encabezados de autorización:** El módulo central de peticiones enviaba encabezados de autenticación con valores nulos o indefinidos cuando el usuario navegaba de forma anónima, generando un comportamiento ambiguo en los servidores.
- **Políticas de seguridad de contenido inexistentes:** El documento principal carecía de políticas de restricción de contenido (CSP), exponiendo a la aplicación a ataques de inyección de código de terceros.

1.4.3. Gateway

- **Elevación de privilegios y falta de control de accesos:** Módulos sensibles, como la administración de roles o el enrolamiento de accesos físicos, carecían de verificación de permisos, permitiendo que usuarios con credenciales básicas ejecutaran acciones administrativas. Asimismo, el enrutador adjuntaba su clave secreta interna sin validar previamente el nivel de acceso del cliente.
- **Ausencia de límites de tráfico y políticas permisivas:** Diversas rutas expuestas carecían de mecanismos para restringir la tasa de peticiones por minuto (Rate Limiting) y utilizaban configuraciones de origen cruzado (CORS) demasiado laxas para entornos de producción.
- **Ejecución de contenedores con privilegios máximos:** El entorno de empaquetado del enrutador ejecutaba el servicio bajo el usuario principal del sistema operativo (`root`), lo que incrementaba el impacto en caso de un compromiso del contenedor.
- **Desactivación de la validación de llaves públicas:** La configuración desactivaba explícitamente la verificación de seguridad al descargar las llaves criptográficas del proveedor de identidades, permitiendo la eventual suplantación de firmas de autenticación.

1.4.4. Microservicio de accesos

- **Almacenamiento en texto plano de secretos criptográficos:** Las semillas secretas utilizadas para validar los códigos dinámicos de acceso fuera de línea se almacenaban en la base de datos sin ningún tipo de cifrado.
- **Fuga de tokens de acceso en registros:** El sistema imprimía los códigos temporales válidos directamente en los registros del servidor, lo que permitiría a un actor interno suplantar identidades en tiempo real.
- **Valores predeterminados rígidos:** Se encontraron identificadores de molinetes/escáners físicos escritos directamente en el código, impidiendo el uso dinámico del sistema en un entorno real con múltiples puertas.
- **Políticas de origen cruzado ausentes:** El servicio carecía de configuración para permitir peticiones seguras desde las aplicaciones integradas de terceros.

1.4.5. Microservicio de analíticas

- **Procesamiento deficiente de mensajes asíncronos:** El receptor de eventos financieros leía los datos entrantes del bus de mensajería sin realizar una validación de estructura previa, lo que podía ocasionar fallos críticos e inconsistencias en las métricas ante un mensaje malformado.
- **Autenticación vulnerable a análisis de tiempo:** El mecanismo de verificación de tokens utilizaba comparaciones de texto convencionales, lo que permitía a un atacante inferir la clave secreta mediante el análisis estadístico de los tiempos de respuesta (Timing Attack).
- **Contratos de interfaz incompletos:** La falta de especificación formal para las respuestas de error en la documentación técnica (OpenAPI) dificultaba la integración automatizada por parte de clientes externos o agentes de Inteligencia Artificial.

1.4.6. Microservicio de autenticación

- **Políticas de contraseñas laxas:** El proceso de registro de nuevos usuarios permitía claves de acceso demasiado cortas, incumpliendo con las recomendaciones internacionales de seguridad para la gestión de identidades.
- **Uso de estructuras de datos genéricas:** Los controladores procesaban la información entrante mediante diccionarios genéricos sin tipo, lo que impedía la autogeneración de esquemas claros y bloqueaba la interacción de herramientas automatizadas que requieren conocer la estructura exacta de los datos.
- **Fuga de detalles técnicos en errores de registro:** Ante un fallo no controlado durante el alta de un usuario, el sistema devolvía al cliente el mensaje de excepción crudo del motor de base de datos o del proveedor de identidad, exponiendo detalles internos de la infraestructura.

1.4.7. Microservicio de bot conversacional (PLN)

- **Falta de validación de firmas de proveedores externos:** El servicio de recepción de mensajes no verificaba criptográficamente las peticiones enviadas por el proveedor de mensajería (Meta), permitiendo la suplantación de identidad mediante el envío de paquetes falsificados.
- **Exposición de datos personales en el servidor:** Se registraban en consola los mensajes y números telefónicos completos de los socios sin procesos de ofuscación, vulnerando las normativas de protección de datos.
- **Caídas por datos no validados:** La lectura directa de la estructura de los mensajes entrantes carecía de controles de seguridad, generando excepciones críticas y denegación de servicio ante mensajes con formatos inesperados.

1.4.8. Microservicio de gestión del club (Core)

- **Exposición de la arquitectura interna de la base de datos:** Ante un fallo interno del servidor, el sistema devolvía al cliente los errores crudos, revelando la estructura exacta de las tablas y campos, lo que facilitaría enormemente el diseño de ataques de inyección de código.
- **Inconsistencia en los contratos de datos para automatización:** Múltiples salidas de información presentaban fallas de validación de esquemas. Esto rompía los contratos prometidos, provocando que cualquier agente de inteligencia artificial externo fuera incapaz de interpretar correctamente la respuesta.

1.4.9. Microservicio de pagos

- **Omisión de autenticación en procesamiento de cobros:** El punto de acceso principal para ejecutar transacciones financieras no aplicaba la validación de la clave interna del sistema, permitiendo a cualquier actor invocar operaciones de cobro de forma anónima.
- **Ausencia de verificación en notificaciones externas:** El servicio procesaba las notificaciones de pago entrantes desde la pasarela externa sin validar su firma criptográfica, exponiendo la plataforma a ataques de saturación o denegación de servicio (DoS).
- **Inseguridad en la comunicación con la base de datos en memoria:** La conexión hacia el intermediario de eventos (Redis) desactivaba la validación de certificados SSL/TLS, abriendo las transacciones financieras a interceptaciones de red o ataques de tipo Man-in-the-Middle.
- **Gestión de credenciales en scripts de migración:** Se hallaron claves secretas débiles asignadas por defecto y cadenas de conexión con contraseñas expuestas dentro de los archivos de configuración de la base de datos.

1.4.10. Panel administrativo web

- **Exposición de credenciales administrativas:** Se detectó que el código fuente realizaba importaciones estáticas de claves privadas de bases de datos, lo que podría comprometer el acceso total a la infraestructura si los archivos se publican por error.
- **Sobrecarga de memoria y fuga masiva de datos:** El sistema descargaba el padrón completo de socios directamente en la memoria del navegador del cliente en lugar de utilizar paginación en el servidor. Esto vulneraba el principio de privacidad al exponer datos masivos y penalizaba gravemente la escalabilidad del sistema.
- **Vulnerabilidad de inyección de código (XSS):** Se encontraron múltiples instancias donde los enlaces a imágenes de productos y documentos de trámites se inyectaban en la interfaz sin un proceso de sanitización previo, permitiendo la ejecución de comandos maliciosos.
- **Manipulación de rutas de red:** Las consultas hacia los servicios internos no codificaban correctamente los identificadores dinámicos, dejando la puerta abierta a ataques de manipulación de rutas (Path Traversal).

1.5. Acciones tomadas

Las observaciones emitidas por el agente automatizado fueron integradas al flujo de trabajo mediante un proceso de refactorización integral en todos los módulos de la aplicación. A continuación, se detallan las correcciones aplicadas y sus fundamentos técnicos.

1.5.1. Cifrado de datos en reposo y gestión segura de credenciales

Se erradicó el uso de contraseñas por defecto y el almacenamiento de claves maestras en el código fuente. Todos los archivos de configuración ahora exigen estrictamente la inyección de credenciales seguras durante el despliegue. Adicionalmente, los secretos utilizados para la validación criptográfica de accesos físicos (TOTP) pasaron a almacenarse con algoritmos de cifrado simétrico, blindando la identidad de los socios ante posibles filtraciones de la base de datos.

1.5.2. Auditoría de dependencias y prevención de ataques de cadena de suministro

Se revisaron exhaustivamente los manifiestos de instalación de todos los servicios para garantizar el uso de librerías oficiales y certificadas, mitigando ataques de confusión de dependencias. Además, se corrigieron problemas de codificación de archivos que provocaban inestabilidades durante la construcción automatizada del software en la nube.

1.5.3. Sanitización de entradas, protección de interfaces y prevención de fuga de datos

Se estandarizó la preparación de peticiones HTTP en todos los clientes web, aplicando codificación de caracteres a todo parámetro dinámico para anular intentos de manipulación de rutas. Se forzó el uso de políticas de seguridad de contenido (CSP) estrictas y funciones de sanitización en todos los recursos multimedia para prevenir ataques de inyección. Por último, se implementó la paginación a nivel servidor, evitando la descarga masiva de datos en los dispositivos de los clientes, protegiendo así la información personal del padrón de socios.

1.5.4. Gestión de concurrencia y robustez de infraestructura

Para soportar de manera segura el tráfico de alta concurrencia, se implementaron sistemas de bloqueo de hilos de ejecución, asegurando la integridad de la memoria en servicios compartidos. Asimismo, se ajustaron las configuraciones de los contenedores de despliegue para deshabilitar las herramientas de recarga en caliente propias del desarrollo y se restringió su ejecución estableciendo usuarios de sistema sin privilegios administrativos (`root`), reduciendo enormemente la superficie de ataque.

1.5.5. Protección criptográfica contra ataques de análisis de tiempo y suplantación

En los módulos de validación de los microservicios se reemplazaron las comparaciones de texto tradicionales por funciones de comparación de tiempo constante. Además, se incorporó la validación obligatoria de las firmas criptográficas emitidas por plataformas de terceros (como las pasarelas de pago y los proveedores de mensajería), garantizando la total autenticidad del emisor antes de procesar cualquier transacción.

1.5.6. Fortalecimiento de los contratos de interfaz para interoperabilidad con IA

Para facilitar la interacción fluida con herramientas externas y agentes de Inteligencia Artificial, se eliminó el uso de estructuras de datos genéricas en los controladores. Todos los datos entrantes y salientes fueron modelados mediante esquemas estrictamente tipados, asegurando su compatibilidad. Se agregaron puntos de conexión específicos con datos puros para los monitores de estado y se reemplazaron los mensajes de error técnicos internos por respuestas estandarizadas que no revelan la arquitectura subyacente.

1.6. Conclusión

La integración de la herramienta OpenClaw como agente de aseguramiento de calidad demostró ser un activo estratégico para el proyecto. El análisis automatizado no solo permitió detectar fallas de configuración habituales en el desarrollo web, sino que también identificó vulnerabilidades lógicas complejas —como el análisis de tiempo, riesgos de cadena de suministro o configuraciones permisivas en contenedores— que suelen ser pasadas por alto en las revisiones de código tradicionales. Gracias a estas correcciones, SocioUnido cuenta con una arquitectura robusta, segura y técnicamente preparada para integrarse de forma transparente con futuras plataformas y agentes inteligentes.